

一种面向蒙特卡洛程序的 128 核可扩展体系结构^{*}

张立, 黎铁军, 张建民

(国防科技大学计算机学院, 湖南 长沙 410073)

摘要: 蒙特卡洛方法是研究粒子输运问题的重要方法, 为蒙特卡洛方法设计定制加速体系结构成为粒子输运模拟的研究热点。分析了一种典型的蒙特卡洛方法代理程序 Quicksilver 的程序特征, 并对存储层次和簇规模等影响可扩展性的结构参数进行了探索, 提出了一种面向蒙特卡洛程序的 128 核可扩展体系结构, 在 128 核配置下实现了相比单核 90 倍的加速比和 70.1% 的扩展效率。

关键词: 蒙特卡洛方法; 粒子输运; 众核体系结构; 可扩展性; 簇

中图分类号: TP302

文献标志码: A

doi: 10.3969/j.issn.1007-130X.2023.04.004

A 128-core scalable architecture for Monte Carlo application

ZHANG Li, LI Tie-jun, ZHANG Jian-min

(College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China)

Abstract: Monte Carlo method is an important method to study particle transport problems. Designing a customized acceleration architecture for Monte Carlo method has become a research hotspot in particle transport simulation. This paper profiles Quicksilver, a typical proxy application using Monte Carlo method, and makes an architectural exploration to the structural parameters that affect scalability, such as storage hierarchy and cluster size. Finally, a 128-core scalable architecture for Monte Carlo program is proposed, which achieves $90\times$ speedup compared to a single core and a scaling efficiency of 70.1% under 128 cores.

Key words: Monte Carlo method; particle transport; manycore architecture; scalability; cluster

1 引言

粒子输运过程广泛存在于各种物理现象中。这里的粒子指的是微观粒子, 如原子、中子、气体分子、光子和各种带电粒子等。在许多领域和前沿工作中都对粒子输运理论提出了需求, 例如核物理、天体物理和医学放射性治疗等。因此, 对粒子输运理论的研究具有十分重要的意义和价值。

蒙特卡洛方法^[1]的基本原理是将问题的解转化为与概率和数学期望相关的统计量, 如问题中随

机事件的概率、随机变量的期望等, 之后通过大量的实验抽样来估计多次实验的观测值, 从而得到问题的解。在中子输运模拟中, 则是对大量中子的运动进行随机模拟, 追踪每个中子的历史轨迹并进行分析, 最后通过统计方法来预测估计量的期望值。蒙特卡洛方法具有几何适应性强、计算精度高和天然的可并行的特点, 被广泛应用于中子输运模拟。在 20 世纪初, 由于计算设备性能的限制, 采用蒙特卡洛方法模拟中子输运过程需要相当长的计算时间, 从而限制了其广泛应用。随着现代高性能计算机的普及, 蒙特卡洛方法成为研究粒子输运问题的

^{*} 收稿日期: 2022-09-24; 修回日期: 2022-11-16

基金项目: 国家重点研发计划(2022YFB2803405); 国家自然科学基金(62072464); 装备预先研究项目(31511010101)

通信作者: 张建民(jmzhang@nudt.edu.cn)

通信地址: 410073 湖南省长沙市国防科技大学计算机学院

Address: College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, Hunan, P. R. China

重要方法之一,并成为国际上的研究热点。然而,粒子输运模拟的精度、规模和复杂性的急剧增加,使得传统的多处理器难以满足其计算规模。因此,对更大规模的众核处理器体系结构,乃至面向蒙特卡洛方法粒子输运问题的领域专用体系结构,提出了进一步的需求。本文基于通用处理器模型提出面向蒙特卡洛程序的可扩展众核体系结构,实现程序关键代码性能随内核规模的基本可线性扩展的目标。

本文所采用的粒子输运模拟程序是 Quicksilver^[2],它是 Mercury^[3]程序的代理程序。Mercury是由劳伦斯利弗莫尔国家实验室 LLNL (Lawrence Livermore National Laboratory)开发的采用蒙特卡洛方法解决粒子输运问题的程序。在该程序中,概率指的是粒子和材料中的原子碰撞时各种事件发生的可能性。这些事件称为反应 (Reactions),包括吸收、散射和裂变等。这些事件发生的概率称为横截面,可以通过实验测量或者从理论上建模。Quicksilver 作为 Mercury 的代理程序,能够较好地捕获 Mercury 的控制流,也就是随机反应和事件带来的分支特征;同时,Quicksilver 也具有 Mercury 查询横截面表时的内存访问特征;最后,在处理大规模问题时,它也能捕获 Mercury 的域分解和节点间通信特征。所以,Quicksilver 能够充分反映 Mercury 程序的主要特征,可以作为研究粒子输运问题的负载程序。

本文的主要工作如下:

(1) 对 Quicksilver 程序性能进行了分析,并展现了 Quicksilver 程序性能特征随 CPU 内核数量增加的变化情况。

(2) 通过对缓存结构、缓存层次、内存类型和 Cluster 规模的探索,提出了一种面向蒙特卡洛程序的可扩展众核体系结构。

(3) 对该体系结构的可扩展性进行了验证并实现了 128 核的系统结构,基本实现了性能的线性可扩展目标,在 128 核时具有约 70% 的扩展效率。

2 相关工作

2.1 粒子输运加速

Tithi 等人^[4]通过私有化冲突严重的数据结构及动态调度实现负载均衡,在英特尔至强处理器 CLX 8280 上将性能提升到了原始版本的 1.8 倍。此外,通过高效地将 Quicksilver 移植到英特尔新

的架构 PIUMA 上,发现 8 核的 PIUMA 芯片的性能是英特尔至强处理器 CLX 8280 (28 核)的 2 倍,且有更好的扩展效率。Fu 等人^[5]通过在体系结构模拟器 gem5 中配置精简内核,牺牲单核的性能且通过并行性来获取面积和功耗的收益,在 4 核精简内核下实现了 $4.45\times$ 的单位功耗性能优势和 $2.78\times$ 单位面积性能优势。Richards 等人^[2]比较了 Quicksilver 在 Nvidia P-100 GPU 和 IBM Power8 CPU 上的性能,其性能受到粒子输运过程中事件发生的比例影响,在碰撞或穿越事件占主导时,GPU 具有更好的性能,但在两者比较均衡的情况下 CPU 更具优势。

2.2 基于 gem5 的多核研究

Gem5^[6]模拟器是一种模块化的平台,常用于计算机体系结构的研究。为了满足高性能系统的需求,单个芯片所包含的计算资源越来越多,然而集成数十或上百个内核可能会引入额外的开销和瓶颈,其原因是多方面的:一是程序本身存在部分只能串行执行的代码段;二是更多的内核数量会对系统的共享资源产生竞争,从而导致额外的开销。因此,目前大多数多核和众核体系结构的研究仅限于有限的核心数。例如,Semakin^[7]估计了一种计算大气层中三维污染传播程序的可扩展性,通过在 gem5 中实现 CPU 模型并配置了 2~32 个核运行程序,得到每个核数下的加速比及扩展效率,结果表明,CPU 的性能和内核数量基本呈线性关系,在 32 核的条件下实现了 $22\times$ 的加速比和 69% 的扩展效率。Butko 等人^[8]测试了 Rodinia 基准测试套件分别在至多 128 核的 big、LITTLE 和 big.LITTLE 内核集群上运行的加速比,其加速比均在约 64 核时趋于平缓。Qureshi 等人^[9]提到 gem5-X 至多支持 256 核的模拟,尽管此工作仅使用到至多 31 核的模拟,仍提升了 gem5 对内核数量的支持。

尽管国内外的研究人员对 Quicksilver 代理程序的加速和众核体系结构的研究取得了一定的成果,但是面向 Quicksilver 这类粒子输运程序的大规模众核体系结构研究相对较少,仍具有较大的研究空间。此外,相比于其它更成熟的蒙特卡洛粒子输运程序^[10,11],Quicksilver 程序诞生的时间更短,也意味着程序具有较多的优化空间。结合软件上并行算法的优化和面向粒子输运问题的硬件设计,可以最大化地加速 Quicksilver 程序运行。

3 Quicksilver 程序特征分析

在实现具体的设计之前,本文首先对 Quicksilver 程序进行一个初步的性能分析。本文主要使用 gem5 生成的统计数据。然而,由于 gem5 统计数据的一些局限性,本文也结合 Intel VTune Profiler^[12]工具在主机上对程序特征进行分析。尽管主机上的体系结构和通过 gem5 建模的体系结构有所差异,但是仍能够体现程序的一些通用特征。

文献[4]对函数 *cycleTracking()* 做了一个比较详细的分析。然而,该文献是基于 28 核 Intel 至强处理器来分析,并没有体现出程序特征随 CPU 核心数量变化的特征。本文使用共享 L2 缓存结构对多核配置下的程序性能进行分析并作为 Baseline。首先对程序在不同 CPU 核心数量下的总体性能进行表征,使用相对单核配置下性能的加速比作为评估指标,其结果如图 1 所示。可以看到,程序在 32 核以内配置下运行时表现出较好的可扩展性,其加速比和 CPU 核心数量基本呈线性关系,本文使用加速比/CPU 核心数来计算其扩展效率,在 32 核下达到了 83.4% 的扩展效率。然而在 64 核下,其加速比明显变缓,其扩展效率仅有 54.4%。影响程序扩展效率的原因很多。首先, *cycleTracking()* 阶段仅对并行块中的粒子进行并行处理,其它部分仍是串行执行,而 CPU 内核数量的增加不能够提升代码串行部分的性能。此外,随着共享 L2 缓存的内核数量增加,会对共享资源产生争用,造成 L2 缓存的访问受限,图 1 中 L2 缓存的平均带宽随内核数量的变化基本和性能趋势保持一致。最后,随内核数的增加,内核之间的通信开销会增加,这也会导致一定的性能损失。

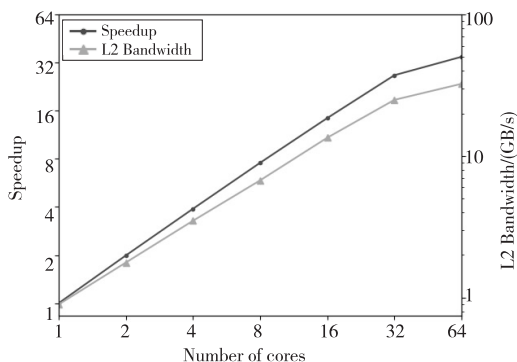


Figure 1 Relative performance and L2 average bandwidths with different numbers of cores

图 1 不同内核数量下的相对性能和 L2 缓存平均带宽

为了进一步研究内核数量增加对 L2 缓存访

问的影响,本文还对 L2 平均命中延迟进行分析,其结果如图 2 所示,对应的坐标轴为最左侧的主坐标轴。可以看到,在 32 核~64 核的过程中,L2 平均命中延迟有一个显著的提升,导致 L2 平均访问时间增加。图 2 中 3 个副坐标轴分别对应 L2 失效率、L2 平均缺失延迟(也就是访存延迟)和 L2 平均访问延迟。在访存延迟基本保持稳定的情况下,L2 平均访问延迟在 32 核以内逐渐下降的原因是 L2 失效率的减小,而在 64 核下 L2 平均命中延迟显著增加,其影响超过 L2 失效率减少所带来的收益,从而导致 L2 平均访问延迟有所增加。L2 失效率减小的原因在于随着 L2 中公有数据被更多的内核所共享,减少了重复存储的数据,从而 L2 缓存的实际有效容量有所增加。

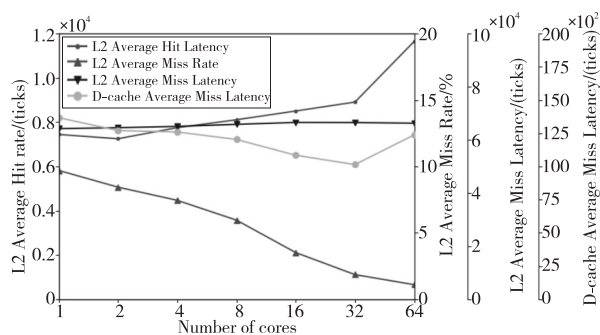


Figure 2 L2 average hit latency, L2 average miss rate, L2 average miss latency and D-cache average miss latency with different numbers of cores

图 2 不同内核数量下的 L2 平均命中延迟、L2 平均失效率、L2 平均缺失延迟和 D-cache 平均缺失延迟

上述分析表明,在内核数量高于 32 核时,L2 缓存成为限制程序进一步扩展的主要因素,主要体现在 L2 的平均访问延迟和 L2 的峰值带宽上。为了研究 L2 缓存对蒙特卡洛程序性能的影响,减少 L2 缓存的访问压力,本文在第 4 节对 L2 缓存结构和 Cluster 配置进行了探索。

由于在 gem5 中得到的统计数据大多是均值数据,对于程序某些特征的变化无法体现,因此,本文结合 VTune 做了一些分析。图 3 是 Quicksilver 程序在主机上运行的内存带宽分布。可以看到,内存带宽在程序执行过程中变化很大,说明程序不同阶段的访存行为有所不同,图中标出了程序运行中的平均带宽和峰值带宽,明显可见峰值带宽远高于平均带宽。对于内核数量极高的系统,内存带宽会显著增加。为了保证内存能满足峰值带宽的需求,本文预先使用能够支持至多 307 GB/s 的带宽的 HMB2^[13]内存。图 4 是 Quicksilver 程序在主机上运行得到的内存访问延迟的分布图。可以看到绝

大多数的内存访问都具有较低的延迟,其中 88% 的内存访问延迟在 10 个周期以内,95% 的内存访问在 20 个周期以内,表明在内存带宽利用率较低的情况下,内存访问并不构成瓶颈。上述分析表明,在内核数量较低的情况下,内存访问并不受限。为此,本文也选择使用 DDR4 内存和 HBM2 内存进行对比,以探索内存的配置。

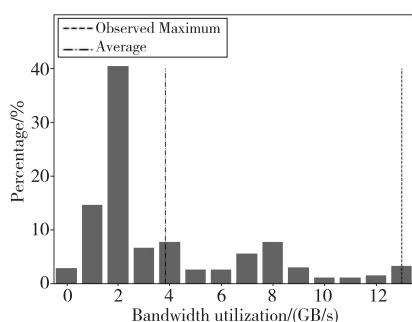


Figure 3 Histogram of bandwidth utilization

图 3 带宽利用分布图

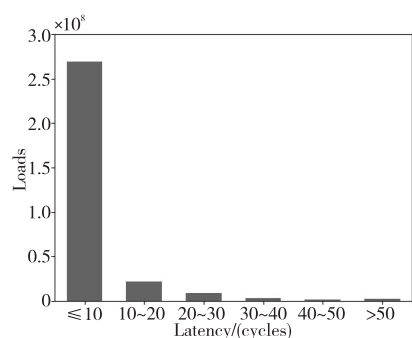


Figure 4 Histogram of memory access latency

图 4 内存访问延迟分布图

在对 CPU 的数据进行分析时,发现每个核的 CPU 时间存在较大的差异,并且这一差异随内核

数增加而增大。图 5 是不同内核规模下每个核运行的周期数,图中柱形高度表示每个核运行的周期数,需要注意的是,每个核开始运行的时刻不一定相同,因此图中最长的运行时间并不代表程序的执行时间;而图中水平直线表示每个核的平均运行周期数,可以看到随着内核数量的增加,运行周期数的平均值和最大值差异也在增大,表明 CPU 的负载不均衡,会影响程序的并行度。针对这一问题,本文分析了 OpenMP 的调度方式对程序可扩展性的影响。

4 面向蒙特卡洛的可扩展众核体系结构探索

HPI (High-Performance In-order)^[14] 内核是 ARM 官方建模的高性能顺序内核。本节使用 HPI 内核作为本文架构的基础,结合第 3 节对程序的分析,对面向蒙特卡洛程序的众核体系结构的存储子系统和计算子系统进行探索。在探索的过程中,本文使用第 3 节中的共享 L2 缓存结构作为基准。首先对存储子系统进行探索,具体如图 6 所示,对多核处理器中共享的二级缓存、是否需要三级缓存以及使用何种类型的内存均进行了探索。对于计算子系统,本文则关注 CPU 聚集数量,即 Cluster 规模。

4.1 二级缓存结构

本文首先在单核配置下对 L2 缓存大小和相联路数进行探索。在这个过程中,L2 的大小分别为 128 KB,256 KB,512 KB 和 1 MB,L2 的相联路

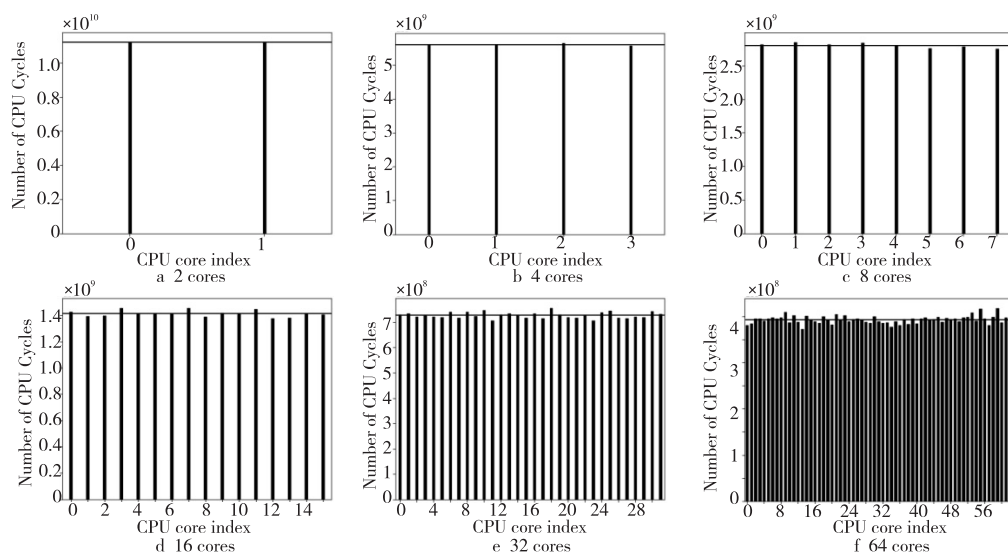


Figure 5 CPU cycles distribution under static scheduling

图 5 静态调度下每个 CPU 核运行周期数分布

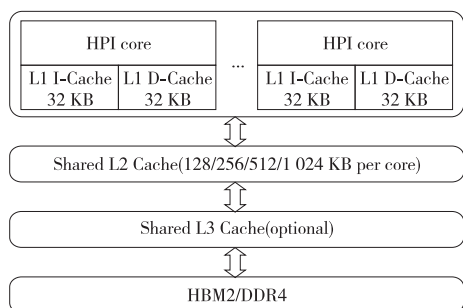


Figure 6 Schematic diagram for exploring the architecture using different parameters

图 6 使用不同参数对体系结构进行探索的示意图

数分别为 2, 4, 8, 16 和 32, 总共 20 组实验, 每组实验使用容量 $\times$ 相联路数标记, 如 128 KB $\times$ 4。通过对 L2 缓存的失效率进行分析, 得出了如图 7 所示的折线图。

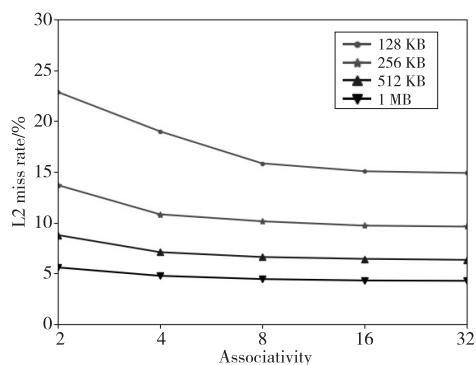


Figure 7 L2 cache miss rates with different numbers of cores, L2 cache sizes, and associated paths

图 7 不同内核数量、L2 大小和相联路数下的 L2 缓存失效率

从图 7 可以得出以下结论。首先, 整体上看, 增加 L2 缓存大小和相联路数都能够减小缓存的失效率。其次, 在 L2 容量较小时, 增加相联路数能够产生较好的效果, 而在 L2 容量较大时, 这一效果则并不明显。这是因为当 L2 容量较小时, 增加相联路数能有效缓解缓存块之间的冲突, 而 L2 容量增加时, 在块的冲突得到明显改善的情况下, 增加相联路数的效果则有所下降。最后, L2 容量的增加在不同相联路数下都对 L2 的失效率有着较为明显的改善。为了确定失效率的减少对程序总体性能的影响, 本文比较了不同配置下相比于基准的性能, 其结果如图 8 所示。

从图 8 可以发现, 将 L2 容量从 128 KB 增加到 256 KB 所得到的性能提升最大。例如, 在 16 路时, L2 容量从 128 KB 增加到 256 KB, 性能提升了 4.4%, 而从 256 KB 增加到 512 KB, 性能仅提升了 2.7%。如果再考虑所增加的 L2 具体容量, 这一差距更为明显。除此之外, 考虑到片上面积和

功耗的因素, 也需要尽可能减少 L2 的容量, 因此本文使用 256 KB 的容量作为 L2 缓存的配置。上述分析表明, 在缓存容量较小时, 增加相联路数能够有效改善 L2 失效率, 这一特征在程序总体性能上也得到了体现。图 8 中, 在 256 KB 配置下, 性能在相联路数达到 16 时几乎不再提升, 因此配置 L2 缓存为 16 路组相联。综上所述, 二级缓存使用 256 KB 的容量和 16 路组相联配置。

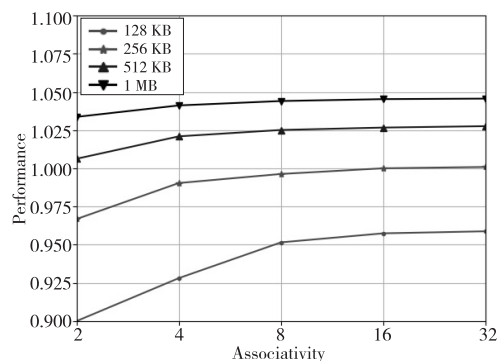


Figure 8 Relative performance with different numbers of cores, L2 cache sizes, and associated paths

图 8 不同内核数量、L2 大小和相联路数下的相对性能

为了验证该配置在不同规模内核下的敏感性, 本文在多核配置下进行了同样的分析和验证, 结果表明多核配置下仍能保持类似的特征。因此, 本文接下来的章节使用本节所确定的 L2 缓存结构。

4.2 缓存层次

为了确认是否需要三级缓存, 本文添加了一个 L3 缓存, 其标签延迟和数据延迟均为 20 个周期, 顺序访问, 相联路数为 16, 大小统一为 16 MB, MSHRs 数量为 32, 具有包容性。为了评估使用 L3 缓存的性能, 本文在使用基准配置之外, 还添加了 2 组额外的配置进行对比。基准配置使用 256 KB $\times$ 16, 其他 3 组配置分别为 512 KB $\times$ 16, 1 MB $\times$ 16 和 256 KB $\times$ 16+L3 缓存。本文使用相对于基准配置的加速比来衡量性能, 其结果如图 9 所示。在图 9 的最后一组配置中可以发现, 使用 L3 所得到的性能提升并不明显, 在单核下具有最高的性能提升, 随着内核数量的增加, L3 对性能的提升反而减少了。这是因为随着内核数量增加, L2 缓存容量也有所增加, L2 缓存失效率降低, 对缓存的丢失具有更好的过滤作用, L3 的作用有所降低。从另一个角度看, 在单核下 256 KB $\times$ 16+L3 配置的性能低于 1 MB $\times$ 16 配置的性能, 表明额外 768 KB 的 L2 缓存能提供比 16 MB 的 L3 缓存更好的性能; 而在 8 核下 256 KB $\times$ 16+L3 配置的性能低于 512 KB $\times$ 16 配置的性能, 表明额外 2 MB 的 L2 缓存比

16 MB 的 L3 缓存具有更好的收益。该结果表明,与其添加 L3 缓存,不如增加 L2 缓存的容量。事实上,本文的实验结果表明,随着内核数量的增加,L3 缓存的命中率有所降低,当 L3 命中率比较低时,仍然存在较多的内存访问,这部分操作会带来不必要的查找延迟,导致 L3 缓存不能带来明显的性能提升。因此,在目前的体系结构中,本文不使用三级缓存。

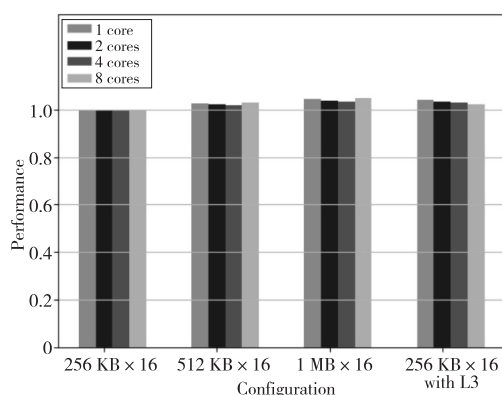


Figure 9 Performance comparison with and without L3 cache

图9 添加 L3 缓存前后的性能对比

4.3 内存类型

第3节预先使用了 HBM2 内存,本节则对众核体系结构的内存类型进行探索,通过对比 HBM2 和 DDR4 的性能,确定内存类型。实验仍然使用 256 KB×16 的基准配置,仅更换内存为 DDR4,DDR4 使用的是 gem5 中已经实现的 DDR4_2400_8×8,内存大小为 4 GB,单通道,理论带宽为 19 200 MB/s。图 10 中柱状图是采用 HBM2 和 DDR4 的性能对比。可以看到,在各个内核规模下,程序总体性能几乎没有差别;进一步分析其平均带宽,如图 10 中折线所示,内存带宽也基本保持一致。

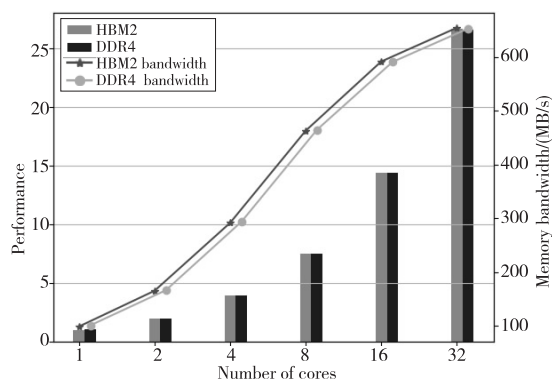


Figure 10 Performance comparison between HBM2 and DDR4

图10 使用 HBM2 和 DDR4 内存的性能对比

可以看到,随着内核数量的增加,内存带宽的增加在变缓,这主要得益于 L2 缓存命中率的提升,实际内存访问次数有所减少。尽管这 2 种内存的平均带宽均不足 700 MB/s,但是在本文最终的 128 核配置中,内存平均带宽达到了 2 607 MB/s。结合第 3 节对内存带宽的分析,表明程序的峰值带宽是平均带宽的 3 倍以上,这意味着在 128 核配置中的峰值带宽可能接近 9 GB/s。虽然 DDR4 目前仍能满足带宽的需求,但是在不影响程序总体性能的情况下,本文仍然选择具有更高带宽的 HBM2 内存,以保证程序在更大内核规模的系统中的可扩展性。

4.4 Cluster 规模

在 gem5 中使用经典内存模型,其默认配置由于端口数量限制只能支持 64 核 CPU 的模拟。为了支持更大规模内核数量的模拟,本文将 gem5-X^[9]中对 Cluster 的支持移植到当前版本的 gem5 中。此外,如第 3 节所述,程序在 64 核配置下无法保持线性的扩展效率,L2 缓存的访问成为限制性能提升的主要因素。为了减少 L2 缓存的访问压力,也需要添加对 Cluster 的支持。图 11 是使用 Cluster 的结构,每个 Cluster 包含一定数量的内核并共享 Cluster 专用的 L2 缓存,若干个 Cluster 和内存直接连接,作为和 Baseline 的对比。本文对 Cluster 的规模进行了调整,通过权衡 L2 缓存的命中率和 L2 命中时的平均访问延迟,来选择最佳的 L2 Cluster 规模。

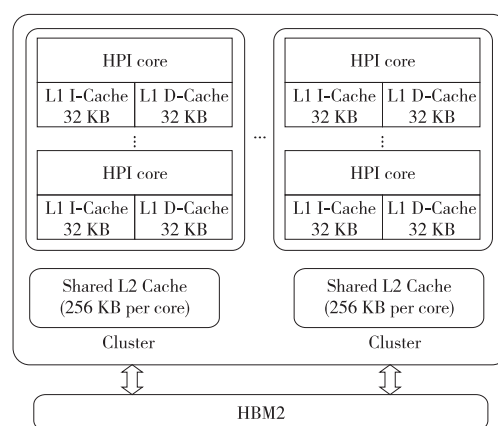


Figure 11 Cluster enabled architecture

图11 启用 Cluster 的结构

本节使用 64 核模拟来研究 Cluster 规模对程序性能的影响,分别配置 Cluster 规模为 1,2,4,8,16,32 和 64。其中,Cluster 规模为 1 即为每个核具有自己私有的 L2 缓存,而 Cluster 规模为 64 则对应着第 3 节中的 Baseline。其运行结果如图 12

所示。可以看到,在 Cluster 规模为 32 的情况下,程序具有最好的性能,其分析和第 3 节中对 L2 平均访问延迟的比较类似,Cluster 规模越大,一方面提高了缓存命中率,另一方面增加了 L2 平均命中延迟,结果表明在 32 核配置下具有最低的 L2 平均访问延迟,这是对 L2 平均失效率和 L2 平均命中延迟的一个较好的权衡。因此,本文选择 Cluster 规模为 32 作为本文配置。

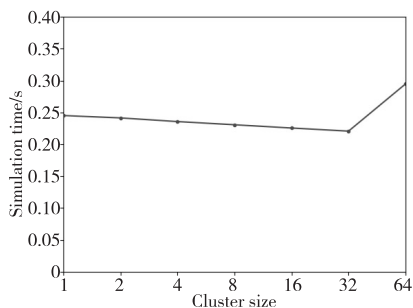


Figure 12 Simulation time with different cluster sizes

图 12 不同 Cluster 规模下的性能

5 面向蒙特卡洛的可扩展众核体系结构设计

经过第 4 节对体系结构的探索,在本节提出了一种面向蒙特卡洛程序的可扩展众核体系结构,如图 11 所示。在现阶段使用 HPI 内核,HPI 内核继承了 gem5 中的 MinorCPU 模型,具有 4 级流水线,其中 Fetch1 阶段负责从指令缓存中读入缓存行或者部分缓存行,并传递到 Fetch2;Fetch2 阶段则将 Fetch1 得到的行存入输入缓存并划分为独立的指令,分支预测器也处于这一阶段;Decode 阶段

进行指令译码并将结果输入到指令向量中供 Execute 阶段使用;Execute 阶段完成指令执行和内存访问功能。关于 HPI 内核的更多细节可以参考 ARM 提供的文档^[14]。在对体系结构的探索过程中,确定 Cluster 规模为 32,因此每 32 个 HPI 内核聚集成 1 个 Cluster,如果内核数量小于 32 个,则所有内核构成 1 个 Cluster,根据内核数量设定好 L2 缓存的大小,最后将 L2 缓存连接到 HBM2 内存。

6 实验与结果分析

6.1 实验设置

本文所确定的面向蒙特卡洛程序的可扩展众核体系结构在 gem5 中的主要参数如表 1 所示。gem5 模拟器运行在服务器上,Quicksilver 源码地址为 <https://github.com/LLNL/Quicksilver>。本文在物理主机上使用 Intel VTune Profiler 作为程序分析工具,主机处理器为 Intel(R) Core(TM) i5-9500 6 核处理器。

6.2 OpenMP 动态调度

在第 3 节中指出,在多核配置下每个 CPU 核的工作时长差异较大,其最长工作时长与平均工作时长差异随内核规模增大而增加,程序出现较为明显的负载不均衡现象。程序总体的运行时间取决于最后一个 CPU 停止工作的时间。因此,通过负载均衡可以使得 CPU 内核在程序并行部分停止工作的时间接近,进而提高其并行度和可扩展性。为了实现负载均衡,将 Quicksilver 程序的并

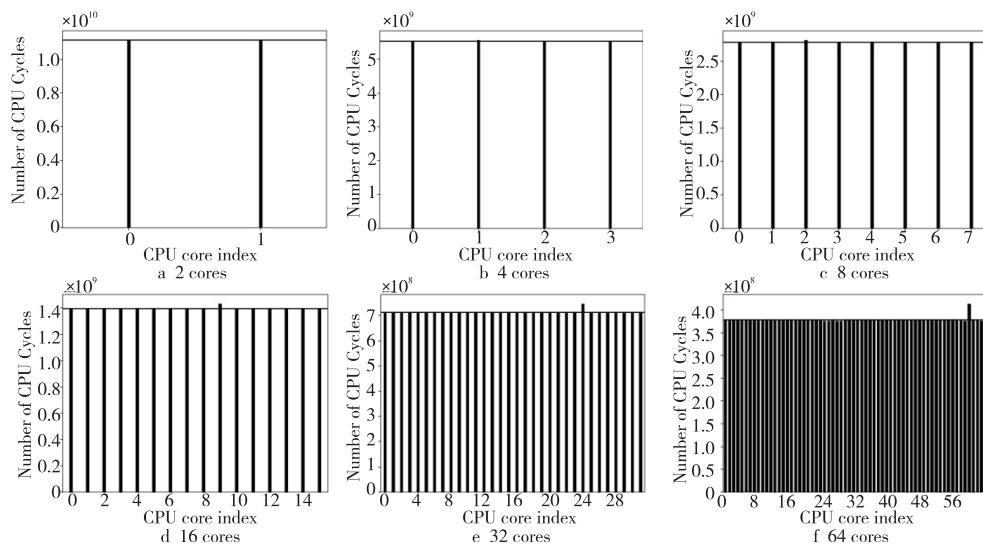


Figure 13 CPU cycles distribution under dynamic scheduling

图 13 动态调度下每个 CPU 核运行周期数分布

Table 1 gem5 simulation parameters

表 1 gem5 模拟参数

参数	配置
CPU type	HPI
CPU frequency/GHz	2.2
Cacheline size/B	64
L1 I-Cache size/KB	32
L1 I-Cache MSHRs	2
L1 I-Cache associativity	1
L1 D-Cache size/KB	32
L1 D-Cache MSHRs	2
L1 D-Cache associativity	2
L2 Cache size/(KB/core)	256
L2 Cache MSHRs	4
L2 Cache associativity	16
Memory type	HBM2
Memory size/GB	4
Memory channels	8
Operating system	Ubuntu 18.04
Cluster size	32

行块中的指导命令修改为 dynamic,从而实现动态调度。图 13 是使用动态调度之后每个核运行的周期数。可以发现,此前 CPU 各个核工作时长差异较大的问题得到了改善,除了其中的一个核,其它所有核的运行周期数基本都保持在平均运行周期数附近。对于运行周期数超过平均周期数的核,其原因是在程序运行中存在串行运行的代码段,而其动态调度策略仅作用于代码中的并行块。

6.3 结果分析

6.3.1 使用 Cluster

图 14 展示了配置 Cluster 前后程序的加速比。可以清楚地看到,在 64 核配置下其性能有一个明显的提升,改善了程序的可扩展性。利用第 3 节扩展效率的计算方法对扩展效率进行定量评估,结果表明,在 64 核配置下实现了 72.7% 的扩展效率,而此前的效率仅有 54.5%。

6.3.2 OpenMP 动态调度

结合 Clutser 配置和 OpenMP 的动态调度,程序的性能和可扩展性进一步得到了提升,其结果如图 15 所示。程序在 64 核配置下的性能比采用静态调度方式时的提升了 17.3%,同时扩展效率提升到了 85.6%。进一步将内核数量增加到 128 核,发现这种策略仍有比较好的可扩展性,在 128

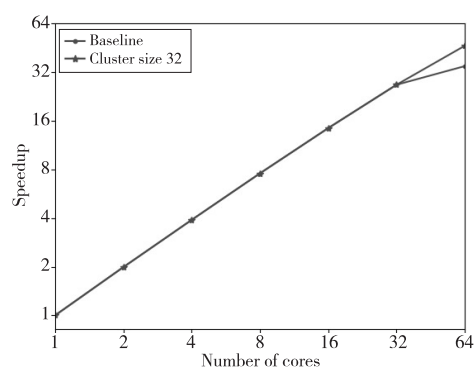


Figure 14 Performance comparison with and without cluster

图 14 采用 Cluster 前后的性能对比

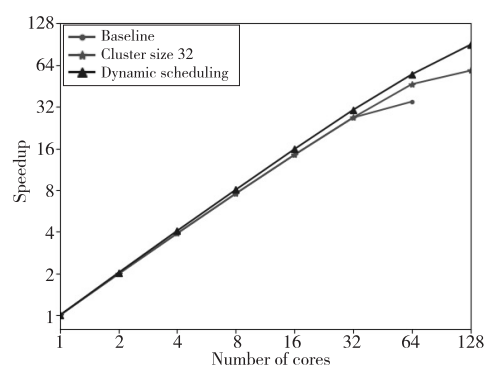


Figure 15 Speedup when using both cluster enabled and dynamic scheduling

图 15 同时使用 Cluster 和动态调度时的加速比

核下达到了 70.1% 的扩展效率,其性能更是比采用静态调度的策略提升了 53.3%。

实验结果表明,结合软件上的调整,对 Quicksilver 程序使用 OpenMP 的动态调度方式能进一步提高其扩展效率。

7 结束语

粒子输运理论对许多物理研究至关重要。随着粒子输运问题规模和复杂性的增大以及对精度要求的提升,对粒子输运问题的模拟加速研究有重要意义。本文面向 Quicksilver 程序,首先分析了不同内核数量下的程序特征,然后对面向蒙特卡洛程序的存储层次的体系结构进行探索,并结合对程序代码的调整,最终实现在 128 核下 90 倍的加速比和 70.1% 的扩展效率。目前使用的是通用的 HPI 内核,后续将对面向蒙特卡洛程序的内核设计进行探索,同时深入分析限制内核规模继续扩大的原因并进行优化,期望将内核规模扩展到至少 256 核。

参考文献:

- [1] Metropolis N, Ulam S. The Monte Carlo method[J]. Journal of the American Statistical Association, 1949, 44(247): 335-341.
- [2] Richards D F, Bleile R C, Brantley P S, et al. Quicksilver: A proxy app for the Monte Carlo transport code mercury[C]//Proc of 2017 IEEE International Conference on Cluster Computing, 2017: 866-873.
- [3] Mercury. [EB/OL]. [2022-05-06]. <https://wci.llnl.gov/simulation/computer-codes/mercury>.
- [4] Tithi J J, Petrini F, Richards D F. Lessons learned from accelerating quicksilver on programmable integrated unified memory architecture (PIUMA) and how that's different from CPU[C]//Proc of International Conference on High Performance Computing, 2021: 38-56.
- [5] Fu S, Li T, Zhang J, et al. MiniMCTAD: Minimalist Monte Carlo transport architecture design[C]//Proc of 2021 IEEE International Conference on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking, 2021: 1-10.
- [6] Binkert N, Beckmann B, Black G, et al. The gem5 simulator [J]. ACM SIGARCH Computer Architecture News, 2011, 39(2): 1-7.
- [7] Semakin A N. Simulation of a multi-core computer system in the gem5 simulator[C]//Proc of XLIV Academic Space Conference, 2021: 090006.
- [8] Butko A, Gamatié A, Sassatelli G, et al. Design exploration for next generation high-performance manycore on-chip systems: Application to big, LITTLE architectures[C]//Proc of 2015 IEEE Computer Society Annual Symposium on VLSI, 2015: 551-556.
- [9] Qureshi Y M, Simon W A, Zapater M, et al. Gem5-X: A many-core heterogeneous simulation platform for architectural exploration and optimization[J]. ACM Transactions on Architecture and Code Optimization, 2021, 18(4): 1-27.
- [10] Briesmeister J F. MCNP-TM-A general Monte Carlo n-particle transport code: LA-13709-M[R]. Version 4C. Los Alamos: Los Alamos National Laboratory, 2000.
- [11] Romano P K, Horelik N E, Herman B R, et al. OpenMC: A state-of-the-art Monte Carlo code for research and development[J]. Annals of Nuclear Energy, 2015, 82: 90-97.
- [12] Intel Vtune Profiler, version 2022[EB/OL]. [2022-05-26]. <https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html>.
- [13] Sohn K, Yun W J, Oh R, et al. A 1.2 V 20 nm 307 GB/s HBM DRAM with at-speed wafer-level IO test scheme and adaptive refresh considering temperature distribution[J]. IEEE Journal of Solid-State Circuits, 2016, 52(1): 250-260.
- [14] Tousi A, Zhu C. ARM system modeling using gem5[R]. Cambridge: ARM Ltd., 2017.

作者简介:



张立(1994-),男,湖北孝感人,硕士生,研究方向为计算机体系结构。**E-mail:** zli20@nudt.edu.cn

ZHANG Li, born in 1994, MS candidate, his research interest includes computer architecture.



黎铁军(1977-),男,湖南邵阳人,博士,研究员,博士生导师,研究方向为高性能计算机体系结构和高性能 CPU 设计。**E-mail:** tjli@nudt.edu.cn

LI Tie-jun, born in 1977, PhD, research fellow, PhD supervisor, his research interests include high performance computer architecture and high performance CPU design.



张建民(1979-),男,山西平遥人,博士,副研究员,研究方向为高性能计算机体系结构和互连网络。**E-mail:** jmzhang@nudt.edu.cn

ZHANG Jian-min, born in 1979, PhD, associate research fellow, his research interests include high performance computer architecture and interconnect network.